مستندات پروژه سیستم مدیریت کتابخانه

مشخصات پروژه

عنوان پروژه: سیستم مدیریت کتابخانه
نوع پروژه: نرم‌افزار تحت وب

Backend: C# ASP.NET Core Web API
Frontend: HTML, CSS, JavaScript
Database: Neon Postgress SQL
ORM: Entity Framework Core
API Documentation: Swagger / OpenAPI

فصل اول: تحلیل سیستم

1-1 مقدمه

کتابخانه‌ها برای مدیریت اطلاعات کتاب‌ها، اعضا و سوابق امانت و بازگشت کتاب‌ها نیاز به یک سیستم منظم دارند. در روش‌های دستی، ثبت اطلاعات در دفتر یا فایل‌های پراکنده می‌تواند باعث افزایش خطا، دشواری جستجو و عدم دسترسی سریع به وضعیت کتاب‌ها شود.

هدف این پروژه طراحی و پیاده‌سازی یک سیستم ساده برای مدیریت اطلاعات یک کتابخانه است که عملیات اصلی مورد نیاز کتابدار را به صورت الکترونیکی انجام دهد.

سیستم طراحی‌شده شامل مدیریت کتاب‌ها، مدیریت اعضا، ثبت امانت و بازگشت کتاب‌ها و ارائه گزارش‌های ساده از وضعیت کتابخانه است.

1-2 معرفی سیستم

سیستم مدیریت کتابخانه یک نرم‌افزار تحت وب است که کاربر اصلی آن کتابدار یا مدیر کتابخانه است.

کاربر می‌تواند:

- کتاب‌های موجود در کتابخانه را مدیریت کند.
- دسته‌بندی کتاب‌ها را مدیریت کند.
- اعضای کتابخانه را ثبت و مدیریت کند.

•کتابی را به یک عضو امانت دهد.

•کتاب امانت‌داده‌شده را بازگرداند.

•سوابق امانت‌ها را مشاهده کند.

•وضعیت کلی کتابخانه را مشاهده کند.

1-3 اهداف سیستم

اهداف اصلی سیستم عبارت‌اند از:

1.ثبت و مدیریت اطلاعات کتاب‌ها.

2.ثبت و مدیریت اطلاعات اعضای کتابخانه.

3.مدیریت فرآیند امانت کتاب.

4.مدیریت فرآیند بازگشت کتاب.

5.نگهداری سوابق امانت‌ها.

6.نمایش تعداد کل کتاب‌ها، کتاب‌های موجود و کتاب‌های امانت‌داده‌شده.

7.کاهش خطاهای ناشی از ثبت دستی اطلاعات.

8.فراهم کردن رابط کاربری ساده برای کتابدار.

1-4 مشکلات روش دستی

در صورت استفاده از روش‌های دستی، مشکلات زیر ممکن است ایجاد شود:

•دشواری جستجوی اطلاعات کتاب‌ها.

•احتمال ثبت اشتباه اطلاعات.

•دشواری تشخیص تعداد نسخه‌های موجود.

•دشواری پیگیری کتاب‌های امانت‌داده‌شده.

•دشواری مشاهده سوابق امانت.

•افزایش زمان مورد نیاز برای انجام عملیات روزمره.

سیستم پیشنهادی با ذخیره اطلاعات در پایگاه داده رابطه‌ای، این عملیات را منظم‌تر می‌کند.

1-5 کاربران سیستم

کاربر اصلی سیستم:

کتابدار / مدیر

وظایف کاربر:

- مدیریت کتاب‌ها
- مدیریت دسته‌بندی‌ها
- مدیریت اعضا
- ثبت امانت
- ثبت بازگشت
- مشاهده گزارش‌های کلی

در محدوده فعلی پروژه، سیستم دارای چند سطح دسترسی پیچیده نیست.

1-6 محدوده سیستم

امکانات موجود در پروژه:

- مدیریت دسته‌بندی کتاب‌ها
- ثبت کتاب
- ویرایش کتاب
- حذف کتاب
- مشاهده کتاب‌ها
- مدیریت اعضا
- ثبت عضو
- ویرایش عضو
- حذف عضو
- مشاهده اعضا
- ثبت امانت
- ثبت بازگشت

- مشاهده سوابق امانت
- گزارش کلی وضعیت کتابخانه

1-7 موارد خارج از محدوده

موارد زیر در نسخه فعلی سیستم پیاده‌سازی نشده‌اند:

- محاسبه جریمه دیرکرد
- رزرو آنلاین کتاب
- ارسال پیامک یا ایمیل
- اپلیکیشن موبایل
- مدیریت چند شعبه
- پنل جداگانه برای اعضا
- سیستم پیشنهاد کتاب
- سیستم پرداخت آنلاین
- مدیریت پیچیده نقش‌ها و دسترسی‌ها

1-8 نیازمندی‌های عملکردی

FR-01: مدیریت دسته‌بندی‌ها

سیستم باید امکان ایجاد، مشاهده، ویرایش و حذف دسته‌بندی کتاب‌ها را فراهم کند.

FR-02: مدیریت کتاب‌ها

سیستم باید امکان ثبت، مشاهده، ویرایش و حذف کتاب‌ها را فراهم کند.

FR-03: مدیریت تعداد نسخه‌ها

سیستم باید تعداد کل نسخه‌ها و تعداد نسخه‌های موجود هر کتاب را نگهداری کند.

FR-04: مدیریت اعضا

سیستم باید امکان ثبت، مشاهده، ویرایش و حذف اعضای کتابخانه را فراهم کند.

FR-05: ثبت امانت

سیستم باید امکان ثبت امانت یک کتاب برای یک عضو را فراهم کند.

FR-06: کنترل موجودی

سیستم نباید اجازه امانت دادن کتابی را که نسخه موجود ندارد بدهد.

FR-07: جلوگیری از امانت تکراری

اگر یک عضو یک نسخه از یک کتاب را در اختیار داشته باشد، سیستم نباید اجازه ثبت مجدد همان کتاب برای همان عضو را بدهد.

FR-08: ثبت بازگشت

سیستم باید امکان ثبت بازگشت کتاب را فراهم کند و موجودی کتاب را افزایش دهد.

FR-09: گزارش کلی

سیستم باید تعداد کل کتاب‌ها، تعداد کتاب‌های موجود، تعداد کتاب‌های امانت‌داده‌شده و تعداد اعضا را نمایش دهد.

1-9 نیازمندی‌های غیرعملکردی

NFR-01: سادگی

رابط کاربری باید ساده و قابل استفاده برای کتابدار باشد.

NFR-02: صحت اطلاعات

سیستم باید از ثبت اطلاعات نامعتبر در عملیات اصلی جلوگیری کند.

NFR-03: یکپارچگی داده‌ها

روابط بین جداول باید توسط کلیدهای اصلی و خارجی کنترل شوند.

NFR-04: کارایی

عملیات معمول مانند مشاهده، ثبت و ویرایش اطلاعات باید در زمان مناسب انجام شوند.

NFR-05: قابلیت نگهداری

ساختار Backend باید به صورت جداگانه از Frontend طراحی شود تا توسعه و نگهداری سیستم ساده‌تر باشد.

NFR-06: امنیت پایه

ارتباط Frontend و Backend از طریق API انجام می‌شود و عملیات اصلی توسط Backend اعتبارسنجی می‌شوند.

1-10 قوانین کسب‌وکار

1. هر کتاب باید دارای عنوان باشد.
2. هر عضو باید دارای اطلاعات شناسایی معتبر باشد.
3. کتاب تنها در صورت وجود نسخه آزاد قابل امانت است.
4. با ثبت امانت، تعداد نسخه‌های موجود یک واحد کاهش می‌یابد.
5. با ثبت بازگشت، تعداد نسخه‌های موجود یک واحد افزایش می‌یابد.
6. هر رکورد امانت باید به یک کتاب و یک عضو مربوط باشد.
7. یک امانت فعال نمی‌تواند دوباره برای همان عضو و همان کتاب ثبت شود.
8. تعداد نسخه‌های موجود نباید بیشتر از تعداد کل نسخه‌ها باشد.
9. تعداد کل نسخه‌ها و نسخه‌های موجود نباید منفی باشند.

1-11 فرآیند اصلی سیستم

روند اصلی استفاده از سیستم به صورت زیر است:

ورود به سیستم
↓
مشاهده Dashboard
↓
مدیریت کتاب‌ها و دسته‌بندی‌ها
↓
مدیریت اعضا
↓
ثبت امانت
↓
کاهش موجودی کتاب
↓
بازگشت کتاب
↓
افزایش موجودی کتاب
↓
مشاهده گزارش

1-12 فناوری‌های مورد استفاده

فناوری	بخش
HTML, CSS, JavaScript	Frontend
C# ASP.NET Core Web API	Backend
Neon Postgress SQL	Database
Entity Framework Core	ORM
Swagger / OpenAPI	API Documentation
REST API / HTTP / JSON	ارتباط

فصل دوم: طراحی پایگاه داده سیستم مورد نظر

2-1 مقدمه

برای ذخیره اطلاعات سیستم از یک پایگاه داده رابطه‌ای استفاده شده است. طراحی پایگاه داده با هدف کاهش افزونگی اطلاعات، حفظ یکپارچگی داده‌ها و ایجاد ارتباط صحیح بین موجودیت‌های سیستم انجام شده است.

پایگاه داده شامل چهار جدول اصلی است:

1.Categories

2.Books

3.Members

4.Borrowings

2-2 جدول Categories

این جدول برای نگهداری دسته‌بندی کتاب‌ها استفاده می‌شود.

فیلد	نوع داده	توضیح
Id	INT	کلید اصلی
Name	NVARCHAR(100)	نام دسته‌بندی

ویژگی‌ها:

- `Id` کلید اصلی و Identity است.
- `Name` اجباری است.

•نام دسته‌بندی باید یکتا باشد.

2-3 جدول Books

این جدول اطلاعات کتاب‌ها را نگهداری می‌کند.

فیلد	نوع داده	توضیح
Id	INT	کلید اصلی
Title	NVARCHAR(200)	عنوان کتاب
Author	NVARCHAR(150)	نویسنده
ISBN	VARCHAR(20)	شماره استاندارد کتاب
CategoryId	INT	کلید خارجی
TotalCount	INT	تعداد کل نسخه‌ها
AvailableCount	INT	تعداد نسخه‌های موجود

قوانین:

```
TotalCount >= 0
AvailableCount >= 0
AvailableCount <= TotalCount
```

2-4 جدول Members

این جدول اطلاعات اعضای کتابخانه را نگهداری می‌کند.

فیلد	نوع داده	توضیح
Id	INT	کلید اصلی
FirstName	NVARCHAR(100)	نام
LastName	NVARCHAR(100)	نام خانوادگی
NationalCode	VARCHAR(10)	کد ملی
Phone	VARCHAR(20)	شماره تلفن
RegisterDate	DATE	تاریخ ثبت عضو

`NationalCode` به صورت Unique تعریف شده است تا دو عضو با یک کد ملی ثبت نشوند.

2-5 جدول Borrowings

این جدول سوابق امانت و بازگشت کتاب‌ها را نگهداری می‌کند.

فیلد	نوع داده	توضیح
Id	INT	کلید اصلی
BookId	INT	کلید خارجی کتاب
MemberId	INT	کلید خارجی عضو
BorrowDate	DATE	تاریخ امانت
ReturnDate	DATE	تاریخ بازگشت
IsReturned	BIT	وضعیت بازگشت

2-6 روابط جداول

روابط اصلی پایگاه داده:

```
Categories
    |
  1 |
    |
  N |
Books
    |
  1 |
    |
  N |
Borrowings
    |
  N |
    |
  1 |
Members
```

بنابراین:

- هر دسته‌بندی می‌تواند چند کتاب داشته باشد.
- هر کتاب می‌تواند چند رکورد امانت داشته باشد.
- هر عضو می‌تواند چند رکورد امانت داشته باشد.
- جدول `Borrowings` تاریخچه ارتباط بین کتاب و عضو را نگهداری می‌کند.

2-7 کلیدهای اصلی و خارجی

کلیدهای اصلی:

```
Categories.Id
Books.Id
Members.Id
Borrowings.Id
```

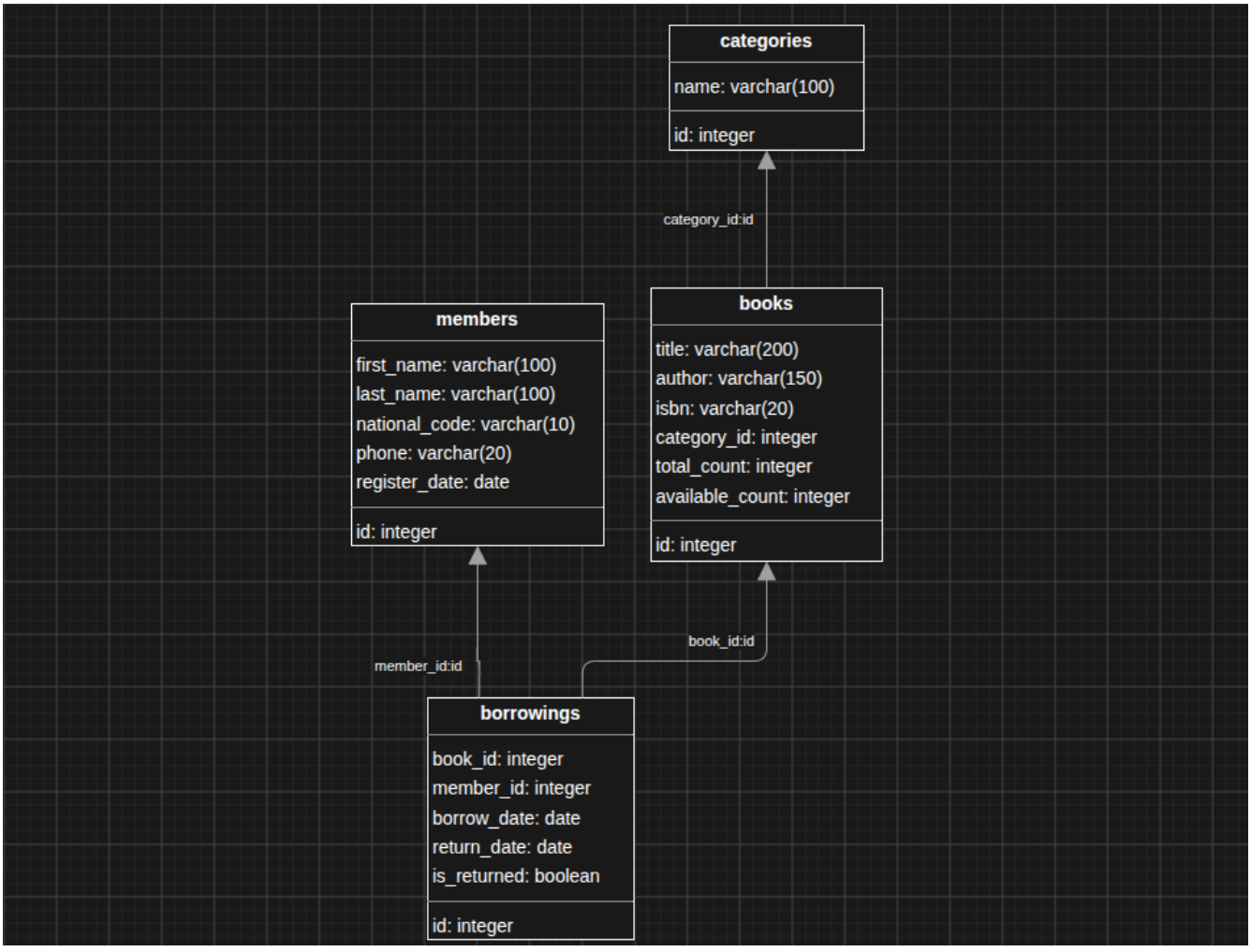
کلیدهای خارجی:

`Books.CategoryId → Categories.Id`

`Borrowings.BookId → Books.Id`

`Borrowings.MemberId → Members.Id`

2-8 نمودار پایگاه داده



در این بخش تصویر ERD/Database Diagram ایجادشده در DataGrip یا ابزار مورد استفاده قرار داده می‌شود.

فصل سوم: پیاده‌سازی پایگاه داده مورد نظر

3-1 ایجاد پایگاه داده

پایگاه داده با نام زیر ایجاد شده است:

```
LibraryManagement
```

سپس جداول مورد نیاز سیستم ایجاد شدند.

3-2 ایجاد جدول Categories

```
CREATE TABLE Categories
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL,
    CONSTRAINT UQ_Categories_Name UNIQUE (Name)
);
```

3-3 ایجاد جدول Books

```
CREATE TABLE Books
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Title NVARCHAR(200) NOT NULL,
    Author NVARCHAR(150) NOT NULL,
    ISBN VARCHAR(20) NULL,
    CategoryId INT NOT NULL,
    TotalCount INT NOT NULL,
    AvailableCount INT NOT NULL,

    CONSTRAINT FK_Books_Categories
        FOREIGN KEY (CategoryId)
        REFERENCES Categories(Id),

    CONSTRAINT CK_Books_TotalCount
        CHECK (TotalCount >= 0),

    CONSTRAINT CK_Books_AvailableCount
        CHECK (AvailableCount >= 0),

    CONSTRAINT CK_Books_AvailableNotMoreThanTotal
        CHECK (AvailableCount <= TotalCount)
);
```

3-4 ایجاد جدول Members

```
CREATE TABLE Members
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    FirstName NVARCHAR(100) NOT NULL,
    LastName NVARCHAR(100) NOT NULL,
    NationalCode VARCHAR(10) NOT NULL,
    Phone VARCHAR(20) NULL,
    RegisterDate DATE NOT NULL DEFAULT GETDATE(),

    CONSTRAINT UQ_Members_NationalCode
        UNIQUE (NationalCode)
);
```

3-5 ایجاد جدول Borrowings

```
CREATE TABLE Borrowings
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    BookId INT NOT NULL,
    MemberId INT NOT NULL,
    BorrowDate DATE NOT NULL DEFAULT GETDATE(),
    ReturnDate DATE NULL,
    IsReturned BIT NOT NULL DEFAULT 0,

    CONSTRAINT FK_Borrowings_Books
        FOREIGN KEY (BookId)
        REFERENCES Books(Id),

    CONSTRAINT FK_Borrowings_Members
        FOREIGN KEY (MemberId)
        REFERENCES Members(Id),

    CONSTRAINT CK_Borrowings_ReturnDate
        CHECK (
            (IsReturned = 0 AND ReturnDate IS NULL)
            OR
            (IsReturned = 1 AND ReturnDate IS NOT NULL)
        )
);
```

3-6 داده‌های اولیه

برای تست سیستم چند دسته‌بندی، کتاب و عضو اولیه در پایگاه داده قرار داده شد.

نمونه دسته‌بندی‌ها:

```
INSERT INTO Categories (Name)
VALUES
(N'برنامه‌نویسی'),
```

```
(N'رمان'),
(N'تاریخ'),
(N'علمی');
```

نمونه کتاب‌ها:

```
INSERT INTO Books
    (Title, Author, ISBN, CategoryId, TotalCount, AvailableCount)
VALUES
    (N'Clean Code', N'Robert C. Martin',
     '9780132350884', 1, 3, 3),

    (N'Design Patterns', N'Erich Gamma',
     '9780201633610', 1, 2, 2),

    (N'شازده کوچولو', N'آنتوان دو سنت اگزوپری',
     NULL, 2, 5, 5);
```

نمونه اعضا:

```
INSERT INTO Members
    (FirstName, LastName, NationalCode, Phone)
VALUES
    (N'امین', N'09120000000' ,'0012345678' ,'قاسمی'),

    (N'علی', N'09121111111' ,'0023456789' ,'احمدی');
```

3-7 ارتباط برنامه با پایگاه داده

Backend با استفاده از Entity Framework Core به SQL Server متصل می‌شود.

کلاس اصلی ارتباط با پایگاه داده:

```
public class LibraryDbContext : DbContext
{
    public LibraryDbContext(
        DbContextOptions<LibraryDbContext> options)
        : base(options)
    {
    }

    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Book> Books => Set<Book>();
    public DbSet<Member> Members => Set<Member>();
    public DbSet<Borrowing> Borrowings => Set<Borrowing>();
}
```

رشته اتصال در فایل تنظیمات برنامه قرار گرفته است.

فصل چهارم: ایجاد فرم‌ها و منوها و کدهای مورد نیاز

4-1 ساختار کلی نرم‌افزار

Frontend با HTML، CSS و JavaScript پیاده‌سازی شده و از طریق REST API با Backend ارتباط برقرار می‌کند.

ساختار Frontend:

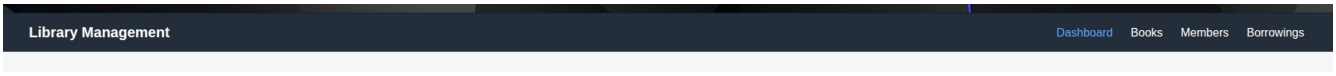
```
LibraryManagement.Web
│
├── index.html
├── books.html
├── members.html
├── borrowings.html
│
├── css
│   └── style.css
│
└── js
    ├── api.js
    ├── dashboard.js
    ├── books.js
    ├── members.js
    └── borrowings.js
```

4-2 منوی اصلی

منوی اصلی سیستم شامل بخش‌های زیر است:

- Dashboard
- Books
- Members
- Borrowings

کاربر می‌تواند از طریق منوی بالای صفحات بین بخش‌های مختلف سیستم جابه‌جا شود.



4-3 صفحه Dashboard

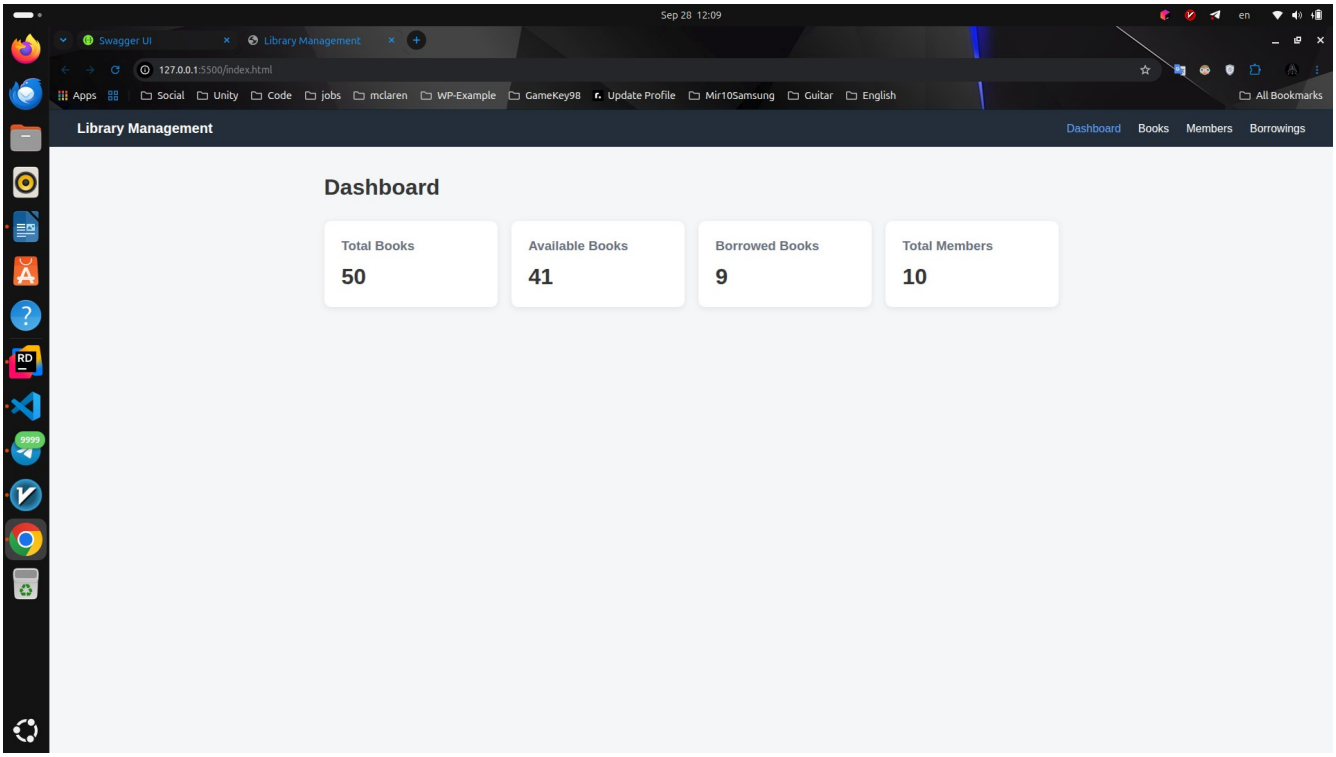
صفحه Dashboard اطلاعات کلی کتابخانه را نمایش می‌دهد.

موارد نمایش داده‌شده:

- Total Books
- Available Books
- Borrowed Books
- Total Members

اطلاعات این بخش از API زیر دریافت می‌شود:

```
GET /api/Reports/summary
```



4-4 فرم مدیریت کتاب‌ها

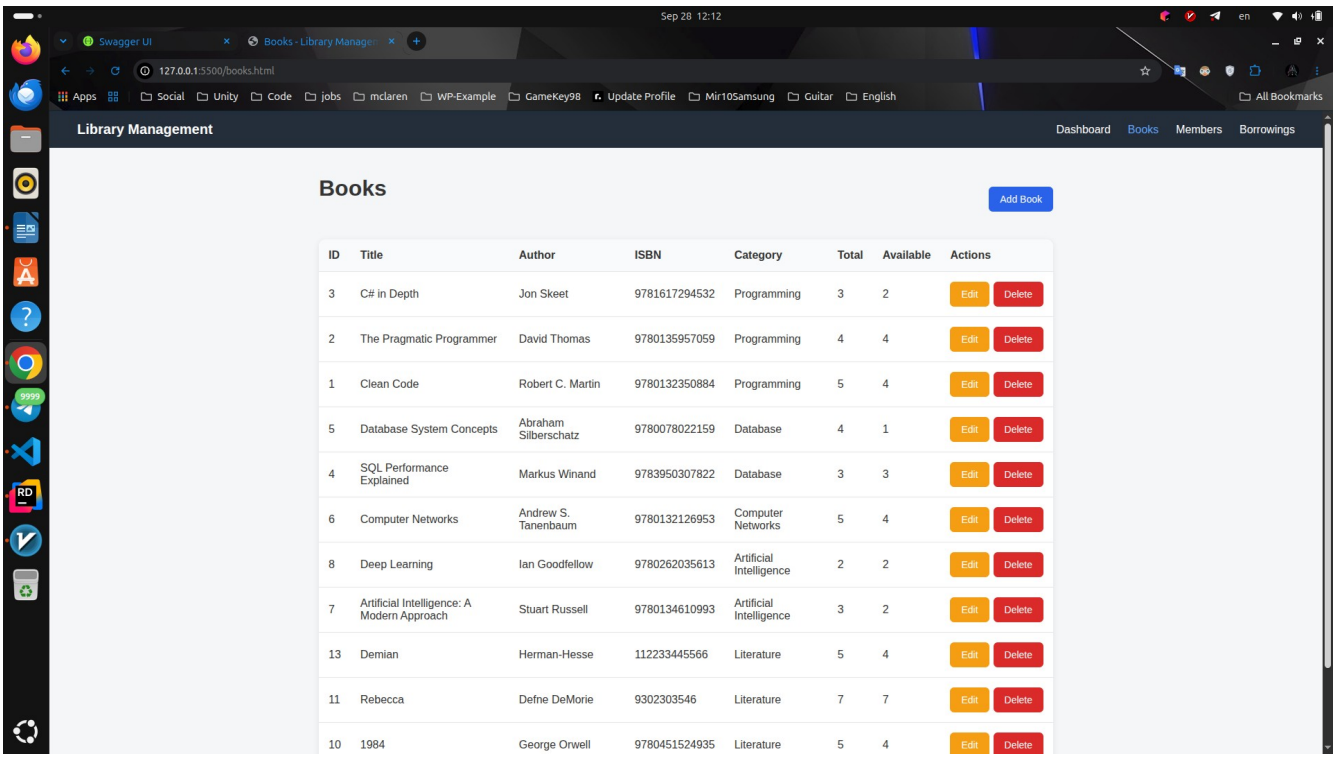
در صفحه Books امکان انجام عملیات زیر وجود دارد:

- مشاهده کتاب‌ها
- افزودن کتاب
- ویرایش کتاب
- حذف کتاب

اطلاعات هر کتاب شامل:

- عنوان
- نویسنده
- ISBN
- دسته‌بندی
- تعداد کل
- تعداد موجود

است.



4-5 فرم مدیریت اعضا

صفحه Members برای مدیریت اعضای کتابخانه استفاده می‌شود.

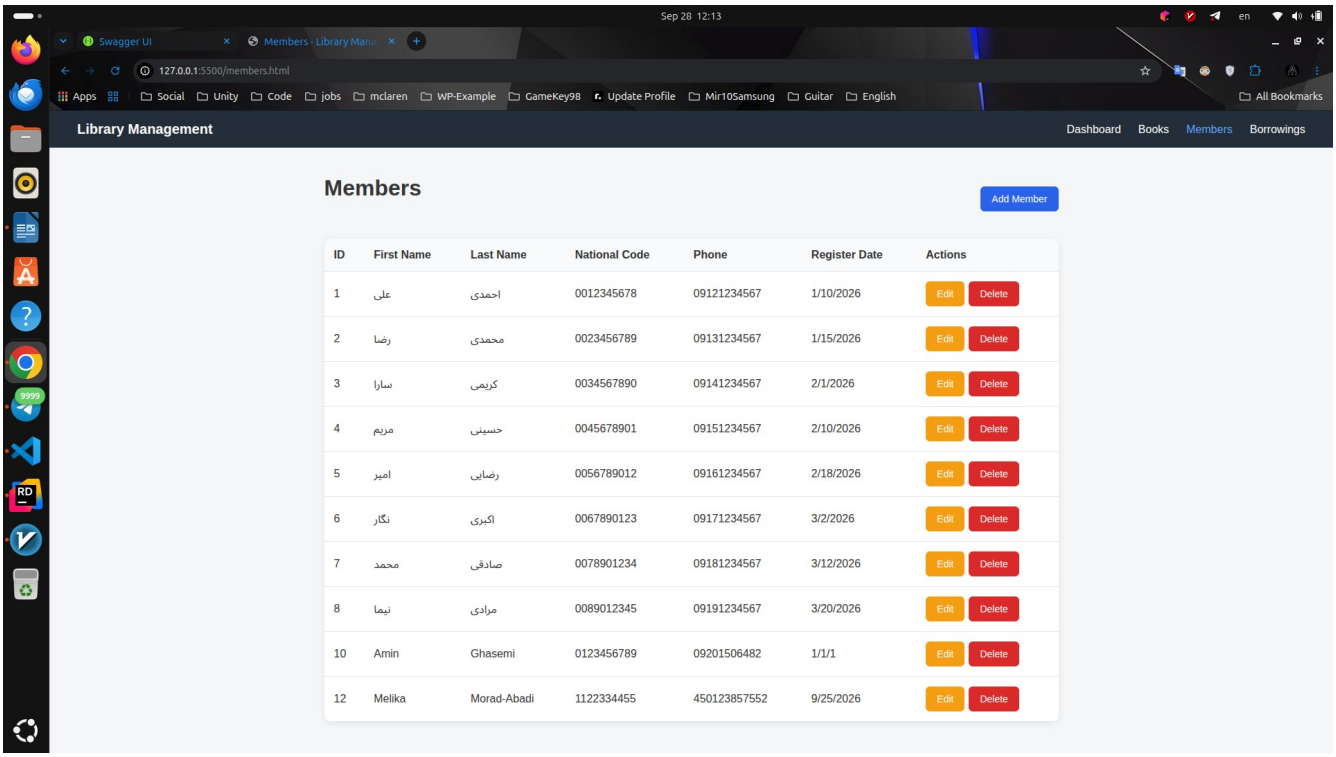
اطلاعات عضو شامل:

- نام
- نام خانوادگی
- کد ملی
- شماره تلفن
- تاریخ ثبت

است.

عملیات:

- ثبت عضو
- ویرایش عضو
- حذف عضو
- مشاهده اعضا



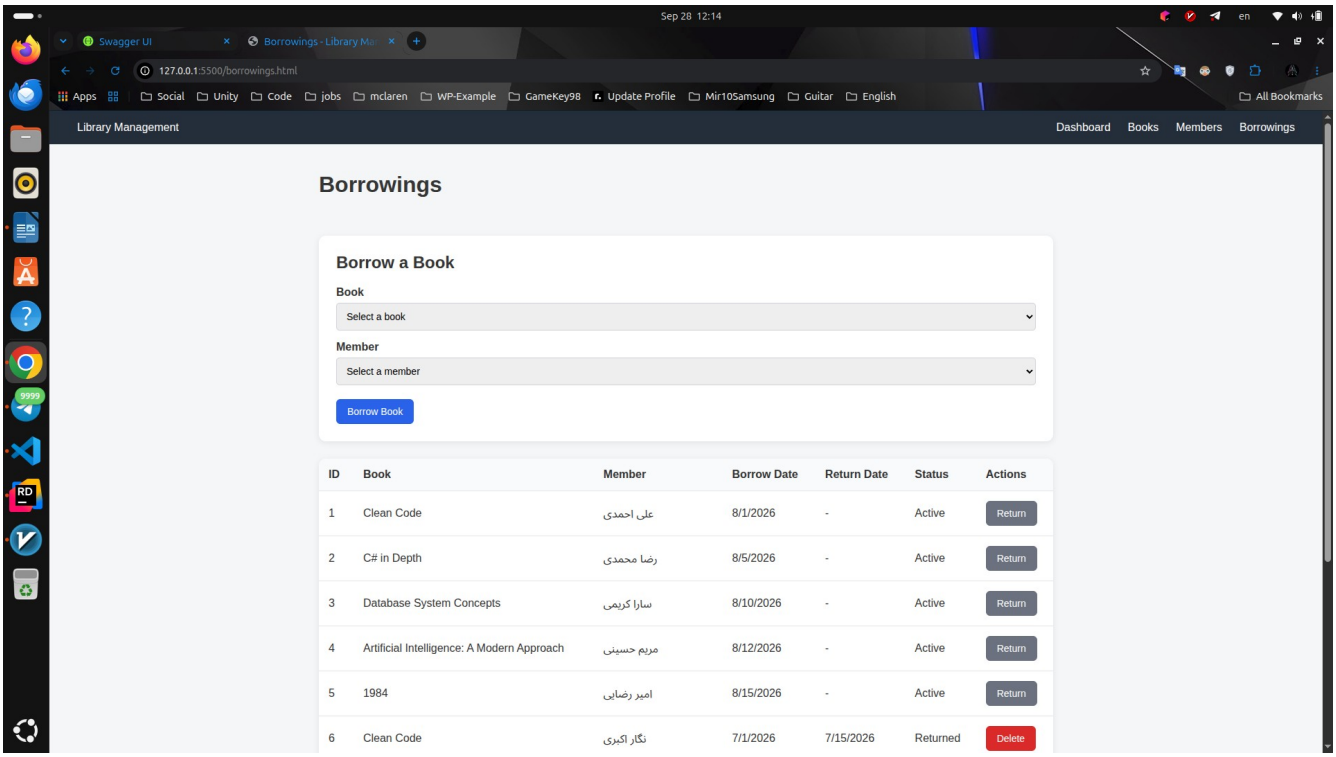
4-6 فرم امانت کتاب

در صفحه Borrowings، کاربر ابتدا کتاب و سپس عضو مورد نظر را انتخاب می‌کند.

پس از ثبت عملیات، Backend موارد زیر را بررسی می‌کند:

1. وجود کتاب
2. وجود عضو
3. وجود نسخه قابل امانت
4. عدم وجود امانت فعال مشابه

در صورت موفقیت، یک رکورد جدید در جدول `Borrowings` ایجاد شده و `AvailableCount` کتاب یک واحد کاهش پیدا می‌کند.



4-7 فرم بازگشت کتاب

برای هر امانت فعال، گزینه Return نمایش داده می‌شود.

با انتخاب Return:

- وضعیت امانت به Returned تغییر می‌کند.

•تاریخ بازگشت ثبت می‌شود.

•تعداد نسخه‌های موجود کتاب یک واحد افزایش پیدا می‌کند.

3	Database System Concepts	سارا کریمی	8/10/2026	-	Active	Return
4	Artificial Intelligence: A Modern Approach	مریم حسینی	8/12/2026	-	Active	Return
5	1984	امیر رضایی	8/15/2026	-	Active	Return
6	Clean Code	نگار اکبری	7/1/2026	7/15/2026	Returned	Delete
7	The Pragmatic Programmer	محمد صادقی	7/10/2026	7/25/2026	Returned	Delete
8	The Little Prince	نیما مرادی	7/20/2026	8/2/2026	Returned	Delete
9	Database System Concepts	رضا محمدی	9/24/2026	-	Active	Return
10	Database System Concepts	نگار اکبری	9/24/2026	-	Active	Return
13	Demian	Melika Morad-Abadi	9/26/2026	-	Active	Return

4-8 کد ارتباط با API

برای جلوگیری از تکرار کدهای ارتباط با Backend، توابع عمومی در فایل `api.js` قرار گرفته‌اند.

نمونه:

```
async function getData(endpoint) {
    const response =
        await fetch(`${API_BASE_URL}${endpoint}`);

    if (!response.ok) {
        const error = await response.text();
        throw new Error(error);
    }

    return await response.json();
}
```

برای ارسال اطلاعات نیز از POST و برای ویرایش از PUT استفاده شده است.

4-9 Backend و API

Backend با ASP.NET Core Web API پیاده‌سازی شده است.

Controllerهای اصلی:

```
CategoriesController
BooksController
MembersController
BorrowingsController
ReportsController
```

نمونه Endpointها:

```
GET    /api/Books
GET    /api/Books/{id}
POST   /api/Books
PUT    /api/Books/{id}
DELETE /api/Books/{id}

GET    /api/Members
POST   /api/Members
PUT    /api/Members/{id}
DELETE /api/Members/{id}

GET    /api/Borrowings
POST   /api/Borrowings
PUT    /api/Borrowings/{id}/return
DELETE /api/Borrowings/{id}

GET    /api/Reports/summary
```

4-10 استفاده از DTO

برای کنترل اطلاعات ورودی و خروجی API از DTOها استفاده شده است.

برای مثال در بخش Borrowings:

```
public class CreateBorrowingDto
{
    public int BookId { get; set; }
    public int MemberId { get; set; }
}
```

این کار باعث می‌شود اطلاعات مورد نیاز عملیات به صورت مشخص از Frontend دریافت شود و از ارسال مستقیم Entityهای پایگاه داده جلوگیری شود.

4-11 کنترل منطق امانت در Backend

در زمان ثبت امانت، موجود بودن کتاب بررسی می‌شود:

```
if (book.AvailableCount <= 0)
    return BadRequest(
        "No available copies of this book."
    );
```

همچنین امانت فعال تکراری برای یک عضو و یک کتاب جلوگیری می‌شود:

```
var alreadyBorrowed =
    await _context.Borrowings.AnyAsync(b =>
        b.BookId == dto.BookId &&
        b.MemberId == dto.MemberId &&
        !b.IsReturned);
```

در صورت ثبت موفق امانت:

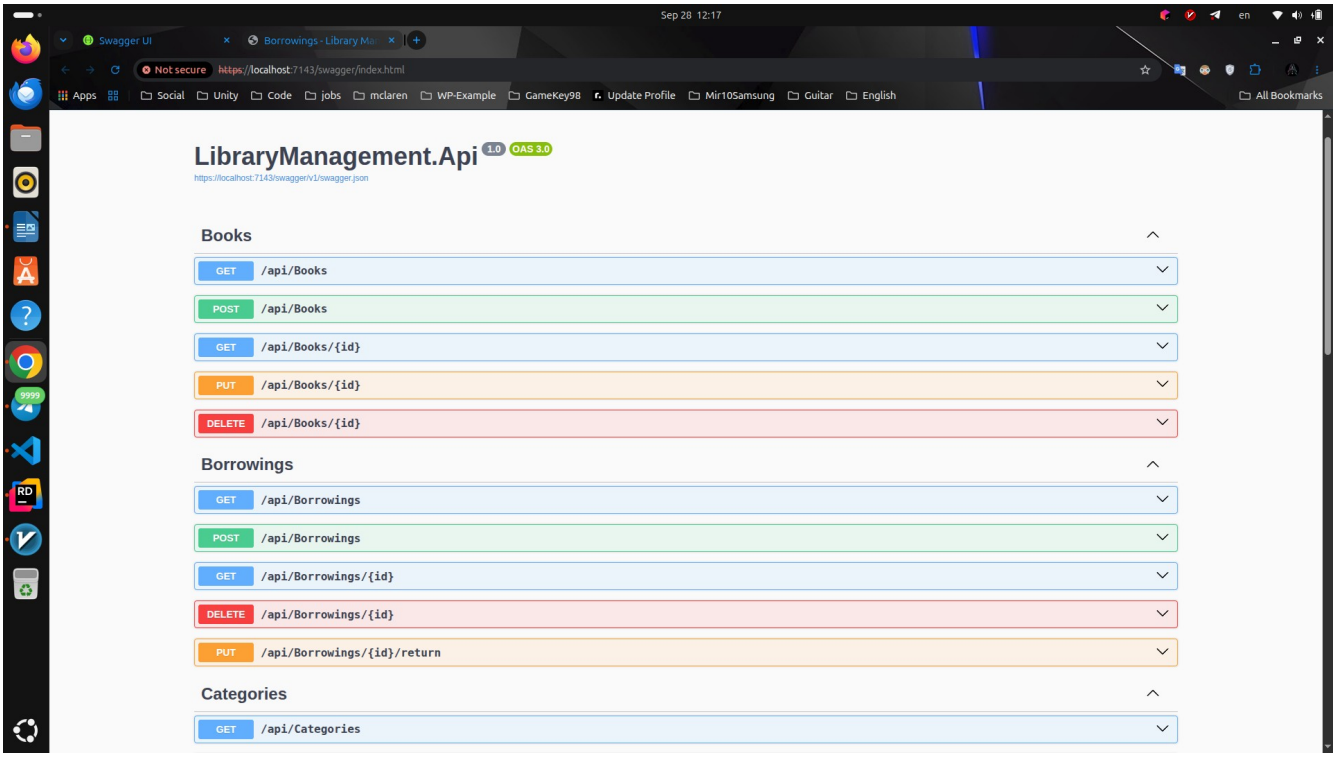
```
book.AvailableCount--;
```

انجام می‌شود.

4-12 مستندات API با Swagger

برای تست و بررسی API از Swagger استفاده شده است.

Swagger امکان مشاهده Endpointها و ارسال درخواست‌های HTTP را فراهم می‌کند.



فصل پنجم: راهنمای استفاده از نرم‌افزار

5-1 اجرای نرم‌افزار

برای اجرای نرم‌افزار ابتدا باید SQL Server و پایگاه داده `LibraryManagement` در دسترس باشند.

سپس پروژه Backend اجرا می‌شود.

پس از اجرای Backend، آدرس Swagger در مرورگر قابل مشاهده است.

Frontend نیز با باز کردن فایل `index.html` قابل اجرا است.

5-2 ورود به Dashboard

پس از اجرای Frontend، صفحه Dashboard نمایش داده می‌شود.

در این صفحه چهار شاخص اصلی نمایش داده می‌شود:

- تعداد کل کتاب‌ها
- تعداد کتاب‌های موجود
- تعداد کتاب‌های امانت‌داده‌شده
- تعداد اعضا

5-3 مدیریت کتاب‌ها

برای مدیریت کتاب‌ها:

1. از منوی اصلی گزینه Books را انتخاب کنید.
2. برای افزودن کتاب روی Add Book کلیک کنید.
3. اطلاعات کتاب را وارد کنید.
4. دسته‌بندی را انتخاب کنید.
5. تعداد کل نسخه‌ها را وارد کنید.
6. روی Save کلیک کنید.

برای ویرایش:

1.کتاب مورد نظر را پیدا کنید.

2.روی Edit کلیک کنید.

3.اطلاعات را تغییر دهید.

4.تغییرات را ذخیره کنید.

برای حذف نیز روی Delete کلیک کنید.

5-4 مدیریت اعضا

برای ثبت عضو:

1.وارد صفحه Members شوید.

2.روی Add Member کلیک کنید.

3.نام را وارد کنید.

4.نام خانوادگی را وارد کنید.

5.کد ملی را وارد کنید.

6.در صورت نیاز شماره تلفن را وارد کنید.

7.اطلاعات را ذخیره کنید.

در همین صفحه امکان ویرایش و حذف اعضا نیز وجود دارد.

5-5 امانت دادن کتاب

برای ثبت امانت:

1.وارد صفحه Borrowings شوید.

2.از قسمت Book یک کتاب انتخاب کنید.

3.از قسمت Member یک عضو انتخاب کنید.

4.روی Borrow Book کلیک کنید.

پس از ثبت موفق:

- رکورد امانت ایجاد می‌شود.
- موجودی کتاب یک واحد کاهش پیدا می‌کند.

•امانت با وضعیت Active نمایش داده می‌شود.

5-6 بازگرداندن کتاب

برای بازگرداندن کتاب:

1.در صفحه Borrowings امانت مورد نظر را پیدا کنید.

2.روی Return کلیک کنید.

3.عملیات را تأیید کنید.

پس از بازگشت:

•وضعیت امانت به Returned تغییر می‌کند.

•تاریخ بازگشت ثبت می‌شود.

•موجودی کتاب یک واحد افزایش پیدا می‌کند.

5-7 حذف سابقه امانت

رکورد امانت فعال قابل حذف نیست.

ابتدا باید کتاب بازگردانده شود.

پس از بازگشت، گزینه Delete برای رکورد نمایش داده می‌شود و کاربر می‌تواند سابقه را حذف کند.

5-8 مشاهده گزارش

برای مشاهده وضعیت کلی کتابخانه، به Dashboard مراجعه کنید.

سیستم اطلاعات زیر را نمایش می‌دهد:

```
Total Books
Available Books
Borrowed Books
Total Members
```

5-9 خطاهای احتمالی

کتاب موجود نیست

اگر تعداد نسخه‌های موجود کتاب صفر باشد، امکان ثبت امانت وجود ندارد.

عضو کتاب را قبلاً امانت گرفته است

اگر یک عضو هنوز کتابی را بازنگردانده باشد، امکان ثبت امانت مجدد همان کتاب برای همان عضو وجود ندارد.

کد ملی تکراری

کد ملی اعضا باید یکتا باشد. در صورت ثبت کد ملی تکراری، پایگاه داده از ثبت عضو جلوگیری می‌کند.

حذف امانت فعال

امانت فعال مستقیماً قابل حذف نیست و ابتدا باید کتاب بازگردانده شود.

5-10 جمع‌بندی

سیستم مدیریت کتابخانه با هدف مدیریت ساده و یکپارچه اطلاعات کتاب‌ها، اعضا و سوابق امانت طراحی و پیاده‌سازی شده است.

سیستم شامل چهار بخش اصلی Dashboard، Books، Members و Borrowings بوده و Backend آن با ASP.NET Core Web API و پایگاه داده آن با Microsoft SQL Server پیاده‌سازی شده است.

با استفاده از این سیستم، عملیات اصلی مورد نیاز یک کتابخانه کوچک شامل مدیریت کتاب‌ها و اعضا، امانت و بازگشت کتاب‌ها و مشاهده گزارش کلی وضعیت کتابخانه قابل انجام است.